

02 | LLM Architecture - Week 05 | Transformers - Summary

28.04.2026

The lecture introduces the **Transformer architecture**, highlighting its role as the core behind modern AI models like GPT and Gemini. It was introduced in the 2017 paper "Attention Is All You Need".

Key Concepts and Architecture:

- **Encoder-Decoder Structure:** The Transformer is fundamentally composed of two main blocks: an **encoder** and a **decoder**.
 - The **encoder** receives input and builds a dense representation (features) of it.
 - The **decoder** uses the encoder's representation along with additional inputs to generate the final output, particularly in a generative setup where it receives outputs from the previous part of the sentence.
 - Both blocks can be used separately or together; a classic use case for the full encoder-decoder is language translation output, particularly in a generative setup where it receives outputs from the previous part of the sentence.
 - Both blocks can be used separately or together; a classic use case for the full encoder-decoder is language translation or summarisation (sequence-to-sequence tasks).
- **Addressing Static Word Embeddings:** Traditional word embeddings (like Word2Vec) are static lookup tables that don't account for multi-semantic meanings of a token or its context.
 - The solution is to create **contextualised embeddings** by selectively integrating information from all neighbouring words.
 - This is achieved using **attention mechanisms**, which determine the importance of different parts of the context for a given token. The

embeddings themselves are learned during the training process, not as a pre-processing step.

- **Transformer's New Concepts:** The lecture introduces several key innovations of the Transformer:
 - **Self-Attention:** Each token attends to all other tokens in the previous layer, breaking away from the directional constraints of RNNs. It makes context-sensitive encodings.
 - **Multi-Head Attention:** Instead of just one attention pattern, multiple attention patterns are used to capture different types of relationships (e.g., semantic, syntactic, long-term, short-term) between tokens. Each head has its own initialised matrices and learns different patterns.
 - **Skip Connections (Residual Connections):** Borrowed from ResNet, these bypass some hidden layers to prevent information loss in deep networks, either by concatenating or adding the direct representation of a token to a higher layer.
 - **Layer Normalisation:** A statistical trick similar to z-score that stabilises training and inference by standardising the output of layers (subtracting mean, dividing by standard deviation). The mean and standard deviation are calculated per batch during training and are constant at inference time.
 - **Dropout Regularisation:** A technique used **only during training** to prevent overfitting by randomly removing some neurons (setting their output to zero). This forces other neurons to learn to compensate, making the network more robust.
 - **Positional Embeddings:** Since self-attention loses the order information of tokens, additional embeddings are added to capture the position of each token within the sequence. These are also learned parameters.

- **Attention Calculation:** Query, key, and value representations are derived by multiplying the input embedding (X) with learned matrices (WQ , WK , WV). The attention score is calculated by multiplying the query with the keys, normalising by the vector size, and applying a softmax function to get probabilities. These probabilities are then multiplied by the values to get the final attention head output.
- **Masked Self-Attention:** In the decoder, to prevent "cheating" by seeing future tokens, masking is applied during training. This zeroes out future token probabilities in the softmax calculation. This is mainly relevant during training; at inference, it processes token by token.

Large Language Models (LLMs) with Transformers:

- LLMs assign probabilities to sequences and generate text by sampling the next token, similar to N-gram models but with a transformer architecture.
- They are trained by learning to guess the next token or a missing token.
- **Applications of LLMs:**
 - **Sentiment Analysis:** By framing a sentence like "The sentiment of the sentence 'X' is:" and letting the LLM complete the next token (e.g., "positive", "negative").
 - **Question Answering:** By adding "Answer:" after a question and letting the LLM predict the answer.
 - **Summarisation:** By adding "TLDR" (Too Long, Didn't Read) after a long text.
 - **Classification:** Wrapping text with start/end tokens for tasks like sentiment analysis or entailment.
- **Types of Transformer-based LLMs:**
 - **Encoder-only Models (e.g., BERT family):** Good for tasks requiring input understanding, like classification, named entity recognition, and part-of-speech tagging. They see the full context and learn masked tokens but are not inherently generative. They are excellent for encoding text into embeddings.

- **Decoder-only Models (e.g., GPT family):** Good for generative tasks like text generation. They predict the next token sequentially, only seeing previous tokens.
- **Encoder-Decoder Models (e.g., T5, BART):** Best for sequence-to-sequence tasks like translation and summarisation.
- **Scaling of LLMs:** The main difference between generations (e.g., GPT-2 to GPT-3/4) is primarily size, meaning more parameters, more computational effort, and more data for training.

LLMs at Inference Time: Sampling

- **Decoding:** The process of choosing a word to generate based on the model's probabilities.
- **Sampling:** The most common decoding method, where tokens are selected based on their probability distribution conditioned on previous choices.
- **Quality vs. Diversity Trade-off:**
 - **High-probability words** lead to higher quality, more accurate, coherent, and factual output but can be boring and repetitive.
 - **Middle-probability words** increase creativity and diversity but can result in less factual or coherent output.
- **Inference Pipeline:** To manage this trade-off, several techniques are applied to the raw probabilities (logits) from the Transformer:
 - **Temperature Scaling:** Applies a scaling factor (temperature, T) to the softmax function.
 - $T = 0$ (or very small): Leads to greedy decoding, favouring only the most probable word. Useful for deterministic outputs in development.
 - $T = 1$: Original distribution (standard softmax).
 - $T > 1$ (large): Makes the distribution more uniform, increasing diversity but potentially generating gibberish.

- **Top-K Filtering:** Only the top k most probable words are considered for sampling. The probabilities of these k words are then renormalised, preventing sampling from the "long tail" of low-probability words.
- **Top-P Sampling (Nucleus Sampling):** Addresses the inflexibility of fixed k in Top-K. It considers only the smallest set of most probable words whose cumulative probability mass exceeds a threshold p. This allows for dynamic vocabulary size depending on the context.

Transformers at Work Visualization:

<https://poloclub.github.io/transformer-explainer/>

In conclusion, the Transformer architecture has revolutionised AI by providing a unified framework for representation, generation, and sequence-to-sequence tasks. LLMs learn everything jointly, allowing for complex tasks to be performed end-to-end at scale, and their outputs are controllable through inference techniques like temperature, top-K, and top-P sampling.